

HO CHI MINH CITY UNIVERSITY OF SCIENCE

Faculty of Information Technology



STATISTICAL LEARNING FINAL PROJECT: TECHQA – TRANSFORMER-BASED QUESTION ANSWERING SYSTEM

Instructor guides

Ngô Minh Minh Nhựt

Lê Long Quốc

Student name	Student ID
Võ Lê Ngọc Hiếu	23127366
Nguyễn Duy Khánh	23127386
Tăng Nhật Minh	23127425
Class: 23CNTThuc	

Table of Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Problem Statement	1
1.3	Project Objectives	2
2	Related Work and Theoretical Background	2
2.1	Retrieval-Augmented Generation	2
2.2	Parameter-Efficient Fine-tuning with QLoRA	4
2.3	Embedding Model and Vector Search	4
3	Dataset	6
3.1	TechQA Dataset Description	6
3.2	Exploratory Data Analysis	7
3.3	Data Preprocessing	7
3.3.1	QA Data Preprocessing	7
3.3.2	Technote Corpus Preprocessing	8
3.4	Train, Validation, and Test Splits	10
4	Proposed Method	11
4.1	System Overview	11
4.2	Document Indexing	12
4.3	Retrieval Module	13
4.4	Generator and Fine-tuning Strategy	14
4.5	End-to-End RAG Pipeline	15
5	Experimental Setup	17
5.1	Implementation Environment	17
5.2	Training Configuration	19
5.3	Evaluation Metrics	19
6	Experimental Results and Analysis	20
6.1	Question Answering Results	20
6.2	Retrieval Results	21
6.3	Qualitative Analysis and Error Analysis	22
6.3.1	Generator Error Analysis	22
6.3.2	Retrieval Error Analysis	22

7	Web-based Application	23
7.1	Application Architecture	23
7.2	User Interface and Main Features	24
7.3	Deployment and Demonstration	27
	Team Contributions	29

List of Figures

1	Overall architecture and inference flow of the TechQA system.	11
2	ChatWindow for submitting questions and reading generated answers. . .	25
3	Sidebar for configuring RAG and selecting sample questions.	26
4	Document Inspector for verifying retrieved evidence.	27

List of Tables

1	Composition and descriptive statistics of the TechQA experimental data .	7
2	Implementation environment and principal software components	18
3	Fine-tuning hyperparameters	19
4	End-to-end question-answering performance on the 160-question TechQA benchmark. Metric values are percentages.	21
5	Retrieval performance on the validation set ($N = 20$)	21

1 Introduction

1.1 Background and Motivation

Technical-support platforms must answer questions involving product-specific terminology, software versions, configuration parameters, and error codes. The information required to solve such problems is usually distributed across long manuals, support articles, and troubleshooting notes. Manually locating the correct passage is time-consuming, while a general-purpose language model may produce a fluent but unsupported answer when the required domain knowledge is absent from its parameters. The TechQA benchmark addresses this practical setting through real technical questions and IBM Technotes [1].

This topic was selected from the three final-project alternatives proposed for the Statistical Learning course. Among those alternatives, TechQA offered an opportunity to study both statistical learning for natural-language generation and information retrieval in a complete, practically demonstrable system. It also presents two complementary challenges: adapting a Transformer to the language and answer style of technical support, and locating useful evidence in a large domain-specific document collection.

To address these challenges, the project combines a fine-tuned Transformer with Retrieval-Augmented Generation (RAG) [2]. QLoRA adapts Llama 3.2-3B-Instruct to TechQA while keeping the computational cost manageable, whereas BGE-M3 and Qdrant provide an external searchable memory of Technotes. At inference time, relevant passages are retrieved and supplied to the generator as evidence. This design aims to produce concise answers that are more relevant, traceable, and grounded than answers generated solely from the model’s internal knowledge.

1.2 Problem Statement

Given a natural-language technical question q and a collection of IBM Technotes D , the system must first retrieve a ranked set of the k most relevant document chunks $D_k \subset D$. It must then generate an answer a conditioned on both the question and the retrieved evidence. Therefore, the end-to-end task can be expressed as

$$D_k = \text{Retrieve}(q, D), \quad a = \text{Generate}(q, D_k). \quad (1)$$

The expected input is an English technical-support question, optionally accompanied by inference settings such as the retrieval mode, the number of documents, and the generation temperature. The output consists of a generated answer, the retrieved source Technotes, their similarity scores, and the measured response latency. Exposing the evidence allows users to inspect whether the answer is supported by the retrieved documents.

The project is scoped to question answering over the TechQA domain rather than open-

domain search or unrestricted Web browsing. Model development uses the answerable TechQA question–answer pairs, and the retrieval component operates on a resource-bounded subset of the IBM Technote corpus represented as 1024-dimensional BGE-M3 vectors in Qdrant. The system supports standalone generation for comparison, but its primary operating mode is RAG-based technical question answering. It does not claim to replace official IBM support; instead, it serves as an experimental assistant for retrieving and summarizing relevant troubleshooting evidence.

1.3 Project Objectives

- Explore, preprocess, and divide the TechQA dataset into training, validation, and test sets.
- Fine-tune Llama 3.2-3B-Instruct with QLoRA for technical question answering.
- Build a retriever using bge-m3 and Qdrant and integrate it into a RAG pipeline.
- Evaluate the models using suitable metrics and compare the experimental configurations.
- Develop a web application consisting of a React frontend and a FastAPI backend.

The resulting deliverables are a reproducible TechQA data-processing workflow, a QLoRA-adapted model published as `AQUABOT/Llama-3.2-3B-TechQA`, a BGE-M3/Qdrant retrieval pipeline, and an end-to-end evaluation on 160 answerable questions. The experiments show that QLoRA improves the standalone model’s token-level F1 from 15.00% to 22.30%, while adding retrieval to the base model improves F1 from 15.00% to 18.83%. The project also delivers a locally runnable web application through which users can configure retrieval, submit questions, read generated answers, and inspect the supporting Technotes and similarity scores. Together, these outputs connect the statistical-learning experiments to a usable technical QA prototype.

2 Related Work and Theoretical Background

2.1 Retrieval-Augmented Generation

The Retrieve-Generate Stages

Retrieval-Augmented Generation (RAG) is an architectural approach that bridges the gap between a Large Language Model’s (LLM) internal knowledge and external, up-to-date, or proprietary data. The process fundamentally consists of two stages:

1. **Retrieve Stage:** When a user submits a query (e.g., a technical question), the system first converts this query into a vector representation. It then searches a pre-built knowledge base (in our case, the TechQA document chunks stored in Qdrant) to find the most contextually relevant documents. This is typically achieved using vector similarity metrics.
2. **Generate Stage:** The retrieved documents (the "context") are concatenated or formatted alongside the original user query. This enriched prompt is then fed into the LLM. The model synthesizes the provided context to generate a coherent, accurate, and context-aware answer.

The Benefits of Grounding

"Grounding" refers to the practice of tying the LLM's responses to verifiable, external sources rather than relying solely on its pre-trained weights. The benefits of grounding include:

- **Mitigating Hallucinations:** By forcing the model to answer based on the provided retrieved text, the likelihood of the LLM inventing incorrect facts (hallucinations) is significantly reduced.
- **Traceability and Trust:** Grounded responses can be accompanied by citations or references to the source documents (e.g., specific TechQA notes), allowing users to verify the information and building trust in the system.
- **Up-to-date Information:** Without retraining the model, the knowledge base can be continuously updated with new documents, ensuring the LLM's answers reflect the latest data.

The Role of RAG for Domain-Specific Knowledge

In the context of the TechQA program, answering technical questions often requires highly specialized, domain-specific knowledge that might not be well-represented in the LLM's general training data. RAG plays a critical role here by acting as an external, specialized memory module. Instead of fine-tuning the LLM on the TechQA corpus—which is computationally expensive and hard to update—RAG allows the system to dynamically inject the exact domain-specific technical notes needed to answer a specific question right at inference time. This ensures that the generated answers are highly relevant to the specific domain and accurately reflect the proprietary or specialized information.

2.2 Parameter-Efficient Fine-tuning with QLoRA

Full-parameter adaptation of a 3.21-billion-parameter language model is computationally prohibitive in the resource-constrained setting of this project. We therefore adopt Quantized Low-Rank Adaptation (QLoRA), which combines a frozen, low-precision backbone with a small set of trainable low-rank matrices [5]. The backbone is loaded using 4-bit NormalFloat (NF4) quantization, a data type designed for approximately normally distributed neural-network weights. Double quantization further reduces the memory footprint by quantizing the quantization constants themselves. Together, these techniques substantially reduce VRAM consumption while retaining the representational capacity of the pre-trained model.

For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA freezes W_0 and learns a rank-constrained update ΔW :

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r}BA, \quad (2)$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$. In our configuration, $r = 16$ and $\alpha = 16$, so the learned update is applied with a scaling factor $\alpha/r = 1$. Adapters are inserted into all seven projection matrices of the attention and feed-forward blocks: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. This broad placement permits domain adaptation in both contextual attention and nonlinear feature transformation while updating only 24,313,856 of 3,237,063,680 parameters (0.75%). Thus, approximately 99.25% of the parameters remain frozen.

Freezing nearly the entire pre-trained backbone also mitigates catastrophic forgetting. The general language-understanding and reasoning abilities acquired during Llama pre-training remain encoded in the unchanged backbone, whereas the low-rank adapters learn the terminology, response style, and troubleshooting structures characteristic of IBM technical support. QLoRA therefore provides an appropriate balance between domain specialization, memory efficiency, and preservation of the base model’s general capabilities.

2.3 Embedding Model and Vector Search

Introduction to BGE-M3

For our knowledge base, we utilize the BGE-M3 (BAAI General Embedding - Multilingual, Multi-granularity, Multi-functionality) model via HuggingFaceEmbeddings. BGE-M3 is a state-of-the-art embedding model designed to handle diverse retrieval tasks. It excels in generating high-quality representations for text across multiple languages and varying text lengths, making it ideal for embedding the complex and varied technical documents found in the TechQA dataset.

Dense Vector Representations

To perform semantic search, text must be translated into a machine-readable format. BGE-M3 transforms our document chunks into dense vector representations (embeddings). A dense vector is an array of continuous floating-point numbers that captures the deep semantic meaning of the text. Unlike sparse representations (like TF-IDF or BM25) that rely on exact keyword matches, dense vectors encode concepts, allowing the system to recognize that "error connecting to DB" and "database connection failure" share similar meanings, even if the exact words differ.

Cosine Similarity and Top-k Retrieval

Once queries and documents are mapped into the same dense vector space, we measure their semantic similarity using metrics such as Cosine Similarity. Cosine similarity calculates the cosine of the angle between two vectors. A value closer to 1 indicates that the vectors point in roughly the same direction, implying high semantic similarity. During the retrieval phase, the system computes the cosine similarity between the query vector and the document vectors in the database. It then returns the documents with the highest similarity scores. This process is known as Top-k Retrieval, where 'k' is the predefined number of most relevant document chunks fetched to form the context for the LLM.

Document Storage and Indexing in Qdrant Cloud

To scale this process for the large TechQA dataset and overcome RAM limitations, our implementation features a robust streaming and batching pipeline before uploading to Qdrant Cloud:

- **Streaming and Text Cleaning:** Instead of loading the entire dataset into memory, we use `ijson` to stream documents. The text is normalized through a `clean_text` step to remove excessive whitespace and formatting issues while preserving technical information. We process these documents in batches (`DOC_BATCH_SIZE = 500`).
- **Section-aware Chunking:** Rather than splitting the whole document blindly, the pipeline is section-aware. It processes distinct sections separately. If sections are unavailable, it falls back to full text. Each section is then independently split using `LangChain's RecursiveCharacterTextSplitter` (`chunk_size = 1000`, `chunk_overlap = 150`). To ensure the LLM understands the context of every chunk, we prepend a metadata header (APAR ID, Title, Section) to each chunk before embedding.
- **Batch Embedding and Deterministic Uploading:** The chunks are embedded using BGE-M3 (`EMBED_BATCH_SIZE = 8`). To make the process fault-tolerant, we generate a deterministic UUID (using `UUID5`) for each chunk based on its source

ID, section index, and chunk index. This enables safe upserts to the Qdrant Cloud collection (techqa_corpus_bge_m3_section_clean). A checkpointing mechanism (techqa_qdrant_section_clean_checkpoint.json) is implemented to save the state after each successful batch upload. This ensures that if the Colab session disconnects, the process can resume seamlessly without starting over or creating duplicate vectors.

- **Indexing and Retrieval:** Qdrant stores these document chunks as payloads alongside their dense vectors. To ensure rapid retrieval across potentially millions of vectors, Qdrant builds an index using the **HNSW (Hierarchical Navigable Small World)** algorithm. This allows Qdrant to perform approximate nearest neighbor (ANN) searches, returning the topk results in logarithmic time (as demonstrated by our client.query_points(query, limit=5) calls).

3 Dataset

3.1 TechQA Dataset Description

TechQA is a real-world benchmark derived from IBM technical-support resources [1]. Each record contains a user question about a concrete software or infrastructure problem, an answerability label (Y or N), and, when answerable, a concise reference answer grounded in an IBM Technote. The task is challenging because questions frequently include specialized product names, versions, libraries, and error codes that are uncommon in general pre-training corpora.

The supervised data contain 600 training questions (450 answerable and 150 unanswerable) and 310 development questions (160 answerable and 150 unanswerable). Fine-tuning retains only records labelled `ANSWERABLE == "Y"`, resulting in 450 training and 160 validation examples; the same 160 answerable development questions are used for generation evaluation. From the approximately 800,000-document Technote corpus, the retrieval pipeline targets a 50,000-document subset for preprocessing, chunking, embedding, and Qdrant Cloud indexing because of the storage and runtime constraints discussed in Section 4.2. Table 1 summarizes the experimental data and corpus scope.

Table 1: Composition and descriptive statistics of the TechQA experimental data

Statistic	Train	Validation	Test benchmark
Number of questions	600	310	160
Answerable questions (Y)	450	160	160
Unanswerable questions (N)	150	150	0
Average question length	15.4 words		
Average reference-answer length	42.6 words		
Median reference-answer length	259 characters		
Full TechQA Technote corpus	Approximately 800,000 documents		
Targeted active knowledge base	50,000 documents		

3.2 Exploratory Data Analysis

The training split contains 450 answerable questions (75.0%) and 150 unanswerable questions (25.0%), whereas the development split contains 160 answerable questions (51.6%) and 150 unanswerable questions (48.4%). Across both partitions, TechQA therefore provides 610 positive and 300 negative examples. The current fine-tuning pipeline excludes the N-labelled records, although they remain useful for future work on abstention and answerability prediction.

Questions average 15.4 words and often contain decisive product names, versions, exception classes, and error codes. Reference answers average 42.6 words and have a median length of 259 characters, indicating that the task generally requires concise extraction of corrective instructions. This motivates the answer-focused training format and conservative decoding strategy. Separately, the retrieval pipeline indexes a targeted 50,000-document subset of the approximately 800,000-document Technote corpus under the available computing and storage constraints. Thus, labelled QA pairs are used to adapt the generator, while the larger unlabelled corpus is processed independently for retrieval.

3.3 Data Preprocessing

3.3.1 QA Data Preprocessing

The QA records are normalized before tokenization to reduce formatting variation without altering technical content. Consecutive whitespace is collapsed, leading and trailing spaces are removed, and anomalous non-informative characters are discarded. Technical symbols, product identifiers, error codes, and punctuation that carry diagnostic meaning

are retained. Records whose answerability annotation is not Y are removed before the Hugging Face datasets are constructed. For each retained record, the user message is formed by concatenating QUESTION_TITLE with the non-empty QUESTION_TEXT, separated by a blank line, and the supervised response is obtained from the stripped ANSWER field. This procedure produces 450 training conversations and 160 validation conversations.

Each normalized example is serialized using Unsloth’s llama-3.1 chat template, which is inherited by Llama 3.2. The system message instructs the model to act as a concise IBM technical-support assistant. During fine-tuning, the user turn contains only the concatenated QUESTION_TITLE and QUESTION_TEXT; no retrieved Technote context is included in the training prompt. The assistant turn contains the reference answer. In schematic form, the serialized sequence is:

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
[system instruction]<|eot_id|>
<|start_header_id|>user<|end_header_id|>
[Question Title]

[Question Text]<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
[reference answer]<|eot_id|>
```

The explicit role delimiters align fine-tuning with the dialogue format expected by Llama-3.2-3B-Instruct, while <|eot_id|> provides an unambiguous turn boundary and later serves as the generation stop token. Retrieved context is introduced only at RAG inference time and therefore remains strictly separate from the supervised fine-tuning input.

3.3.2 Technote Corpus Preprocessing

To ensure high-quality retrieval and accurate generation, the IBM Technote corpus undergoes a dedicated preprocessing pipeline before embedding and indexing. This pipeline transforms raw, loosely structured records into highly contextualized vector points.

Memory-Efficient Streaming Given the large size of the TechQA corpus, loading the entire JSON file into memory at once is inefficient and prone to out-of-memory errors. To address this, the pipeline utilizes a streaming parser (`ijson`) to yield and process documents iteratively in batches. This ensures a low, constant memory footprint regardless of the dataset’s total size.

Document Cleaning Raw technical notes often contain inconsistent formatting. The `clean_text` function normalizes the text by:

- Replacing carriage returns (`\r\n`, `\r`) with standard newlines.
- Stripping leading and trailing whitespace from every line.
- Collapsing multiple consecutive blank lines into a single blank line to preserve paragraph boundaries without wasting tokens on empty space.
- Normalizing repeated spaces or tabs within lines into a single space. This light cleaning ensures that critical technical information is preserved while formatting noise is removed.

Section-Aware Processing and Filtering Rather than treating an entire APAR document as a single continuous block of text, the pipeline extracts content iteratively section by section.

- **Filtering Criteria:** Sections that are completely empty or missing valid content are automatically skipped.
- **Fallback Mechanism:** If a document has no valid sections, the pipeline falls back to using the full document text as a single "FULL_DOCUMENT" section.

Text Chunking for Retrieval To fit within the embedding model's optimal context length and the LLM's prompt limitations, each valid section is chunked independently.

- **Chunking Parameters:** We use LangChain's `RecursiveCharacterTextSplitter` configured with a `chunk_size` of 1000 characters and a `chunk_overlap` of 150 characters.
- **Semantic Boundaries:** By splitting sections independently, we guarantee that chunks do not arbitrarily cross over semantic boundaries (e.g., mixing a bug description with a patch instruction).

Metadata Preservation and Prepending To provide maximum context to the LLM when a single chunk is retrieved out of isolation, metadata is preserved both structurally and textually:

- **Header Prepending:** A header is injected at the top of every chunk's text content (containing APAR ID, Title, and Section). This allows the LLM to instantly understand what document and section the chunk belongs to without needing to parse external JSON fields.
- **Payload Metadata:** The Qdrant payload natively preserves structural metadata such as `id`, `source_id`, `title`, `section`, `section_index`, and `chunk_index`.

Resiliency via Checkpointing and Deterministic IDs Processing large corpora is susceptible to network interruptions or environment crashes. To provide a fault-tolerant ingestion process, the pipeline implements a document-level checkpointing system. Furthermore, to prevent duplicate vectors when resuming or re-running a batch, the system computes a deterministic UUID (UUID5) for each chunk. This UUID is generated using a stable combination of the `source_id`, `section_index`, and `chunk_index`. If a batch is re-uploaded, Qdrant performs a safe upsert (overwriting the existing point with the same ID) rather than creating duplicate entries.

Final Format for Indexing The final processed chunk is structured as a Qdrant `PointStruct` ready for uploading, containing:

- **id:** The deterministic UUID5 string.
- **vector:** The 1024-dimensional dense vector generated by the BGE-M3 model.
- **payload:** A JSON object containing the `page_content` (the text with the prepended header) and the structured metadata dictionary.

3.4 Train, Validation, and Test Splits

The original TechQA files contain 600 training questions and 310 development questions. After retaining only records labelled `ANSWERABLE == "Y"`, the implemented experimental protocol uses the data in three operational roles:

- **Training set:** The 450 answerable questions from the official training partition are used for supervised fine-tuning. These examples contribute to the response-only language-modeling loss and are the only QA records used to compute gradients and update the LoRA parameters.
- **Validation set:** The 160 answerable questions from the official development partition are supplied to `SFTTrainer` as the evaluation dataset. Validation loss is measured after each training epoch to monitor optimization behaviour. These examples do not contribute to gradient computation or parameter updates.
- **Test benchmark:** The labels for the official TechQA test set are not publicly available because it is a blind IBM evaluation set. Consequently, after fine-tuning is complete, the same 160 answerable development questions are reused as the practical benchmark for the final quantitative comparison of the four LLM/RAG configurations reported in Section 6.1.

The terms *validation set* and *test benchmark* therefore describe two successive experimental roles for the same 160 development questions, rather than two disjoint subsets. The benchmark

remains unseen by the gradient-based optimization procedure, but it is not an independent official test partition because its loss is observed during training. This limitation is stated explicitly to avoid overstating the statistical independence of the final evaluation. The reference answers are never inserted into the Qdrant index; retrieval accesses only the external Technote corpus. The random seed 3407 is used for LoRA initialization and other stochastic training operations.

4 Proposed Method

4.1 System Overview

The proposed system is organized as a layered Retrieval-Augmented Generation architecture that separates user interaction, API orchestration, retrieval, and answer generation. This separation is reflected directly in the project structure: the React application is contained in `frontend/`, the REST service and request schemas are implemented in `backend/`, and the machine-learning components are isolated in `engine/`. Consequently, each layer can be tested or replaced without changing the external API contract.

Figure 1 summarizes the end-to-end data flow. A user enters a technical-support question in the React interface together with optional settings such as retrieval mode, Top- k , and generation temperature. The frontend sends a JSON request to `POST /api/ask`. FastAPI validates this request with a Pydantic `QuestionRequest` model and delegates it to the `QAService`, which serves as the boundary between the web layer and the RAG engine.

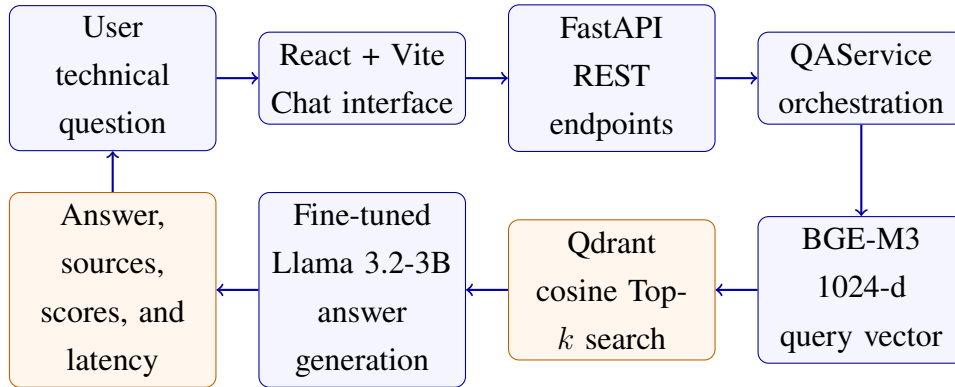


Figure 1: Overall architecture and inference flow of the TechQA system.

Within the engine, `BGEM3Embedder` converts the question into a 1024-dimensional, L2-normalized dense vector. The `QdrantVectorStore` performs cosine-similarity search over the configured TechQA collection and returns the highest-ranked chunks with their payload metadata. The `RAGPipeline` concatenates the retrieved chunk contents into a reference-context block and combines it with the original question using the Llama 3 chat

template. The `LlamaGenerator` then loads either the locally exported model directory or the configured Hugging Face checkpoint and produces the final answer. In direct-LLM mode, the retrieval stage is skipped and the generator answers from its fine-tuned parametric knowledge alone.

The response returned to the frontend contains more than the generated text. It also includes the ranked source documents, document identifiers, titles, similarity scores, retrieval mode, model information, and measured latency. This structured output supports the Document Inspector and enables users to compare an answer against its retrieved evidence. The API additionally provides health, collection-statistics, indexing, sample-question, and evaluation-metric endpoints, allowing the same application to support both demonstration and diagnostic workflows.

The implementation also includes a graceful-degradation path. If the external Qdrant service, embedding model, or full language model cannot be initialized, `QAService` falls back to a small in-memory TechQA demonstration corpus. Documents in this corpus are ranked using keyword, title, and content overlap, while the response schema remains unchanged. This fallback is not used as a substitute for the reported full-corpus experiments; its purpose is to keep the web application operational on machines without sufficient GPU or vector-store resources.

4.2 Document Indexing

Loading and Chunking

Due to the large size of the TechQA corpus (approximately 800,000 documents), documents are loaded using a memory-efficient streaming approach with the `ijson` library. Technotes are processed in manageable batches (`DOC_BATCH_SIZE = 500`).

Targeted Corpus Scope: While the full TechQA corpus contains approximately 800,000 documents, our program explicitly limits the active knowledge base to a targeted subset of 50,000 documents. This operational design choice is directly driven by practical infrastructure constraints: the pipeline operates on the free tier of Google Colab (utilizing the T4 GPU) and the free tier of Qdrant Cloud. Processing and indexing the entire 800,000-document corpus with 1024-dimensional dense vectors would quickly exceed Colab’s session time limits and Qdrant Cloud’s free-tier storage quotas (typically 1GB RAM/Disk). By bounding the scope to 50,000 documents, the knowledge base remains operationally robust and highly relevant while strictly adhering to these cloud resource limitations.

The chunking process is section-aware; each distinct section (e.g., `ERROR DESCRIPTION`, `LOCAL FIX`) is independently split using `LangChain’s RecursiveCharacterTextSplitter` with a `chunk_size` of 1000 characters and a `chunk_overlap` of 150 characters. This ensures that chunks do not arbitrarily cross semantic boundaries.

Embedding Generation

Once chunked, the text is converted into dense vector representations. We utilize the BGE-M3 model (BAAI/bge-m3) via HuggingFaceEmbeddings. The model outputs 1024-dimensional embeddings for each chunk. Batched embedding (EMBED_BATCH_SIZE = 8) on a GPU is used to accelerate this computationally intensive process, producing high-quality vectors that capture the deep semantic meaning of the technical notes.

Storage and Updating in Qdrant

The generated 1024-dimensional vectors and their corresponding textual metadata are stored in a Qdrant Cloud collection. To ensure fault tolerance and safe updates:

- **Deterministic IDs:** A stable UUID (UUID5) is generated for each chunk based on its source APAR ID, section index, and chunk index.
- **Upsert Mechanism:** When a batch is uploaded, Qdrant uses an upsert operation. If a chunk's ID already exists in the collection (e.g., when resuming from a failed batch upload), its vector and metadata are safely updated rather than duplicated.
- **Payload:** The chunk's text (prepended with context headers) and structured metadata are stored within the Qdrant point's payload, keeping all necessary information linked directly to the vector.

4.3 Retrieval Module

Question Embedding and Similarity

When a user submits a query, the Retrieval Module first passes the raw text query through the exact same BAAI/bge-m3 model to produce a 1024-dimensional query vector. Crucially, `normalize_embeddings=True` is used to ensure all vectors are L2-normalized. The similarity between the query vector and the document vectors in the Qdrant index is measured using Cosine Similarity, which evaluates the semantic alignment between the two points in the high-dimensional space.

Top-k Selection and Score Thresholds

The system uses Qdrant's native `query_points` API to perform an approximate nearest neighbor (ANN) search.

- **Top-k:** The retrieval is configured to select the most relevant chunks by setting a hard limit (e.g., `limit=5`).

- **Score Thresholds:** Along with the payload, Qdrant returns a cosine similarity score for each matched point. These continuous scores (ranging up to 1.0) allow the system to apply a score threshold in the future, filtering out low-confidence results if the query is out-of-domain or too ambiguous.

Context Format for Generation

The retrieved points contain the `page_content` in their payloads. Because of our preprocessing pipeline, this content is not just an isolated snippet. It is formatted with a prepended header, yielding the following context format:

```
APAR ID: [Source ID]
Title: [Technote Title]
Section: [Section Name]
[Chunk Text Content...]
```

This structured context is concatenated and passed directly to the generator LLM. The explicit metadata headers guarantee that the LLM is fully aware of the document origin and logical section of every retrieved chunk, maximizing the accuracy and traceability of the final answer.

4.4 Generator and Fine-tuning Strategy

The generator is initialized from `unsloth/Llama-3.2-3B-Instruct`, a 3.21-billion-parameter instruction-tuned decoder-only language model from Meta’s Llama 3.2 family [6]. Its architecture employs rotary positional embeddings (RoPE) to encode relative token positions and grouped-query attention (GQA) to reduce the key-value cache and improve inference efficiency. We refer to this unchanged checkpoint as the *base model*. The domain-adapted checkpoint produced by our training pipeline is published as `AQUABOT/Llama-3.2-3B-TechQA` and is referred to as the *fine-tuned model*.

Training follows the QLoRA formulation introduced in Section 2.2. The frozen backbone is represented in 4-bit NF4 precision with double quantization, while rank-16 adapters with $\alpha = 16$ and zero adapter dropout are trained on `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. The training log reports 24,313,856 trainable parameters out of 3,237,063,680 in total (0.75%), leaving approximately 99.25% unchanged. The trainable low-rank parameters specialize both the attention mechanism and the MLP blocks for IBM terminology and concise technical-support responses without overwriting the model’s general linguistic and reasoning knowledge.

Each training instance is encoded with the Llama 3 chat template described in Section 3.3.1. The notebook applies Unsloth’s `train_on_responses_only` transformation, which masks the system and user tokens so that only the assistant response contributes to the loss. Let $y_{1:T}$ denote the tokenized sequence, $m_t \in \{0, 1\}$ the response-token mask, and θ

the trainable adapter parameters. Supervised fine-tuning minimizes

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{t=1}^T m_t \log p_{\theta}(y_t | y_{<t}), \quad (3)$$

where $m_t = 1$ only for assistant-response tokens. This objective concentrates the gradient signal on answer generation rather than learning to reproduce the prompt. Sequences are truncated or padded to a maximum length of 2,048 tokens. After training, the LoRA update is merged into the backbone and exported in 16-bit precision. The merged checkpoint is published as `AQUABOT/Llama-3.2-3B-TechQA` and can be evaluated with or without retrieved context.

4.5 End-to-End RAG Pipeline

The Retrieval-Augmented Generation (RAG) architecture of the system is designed as an end-to-end processing pipeline. It tightly integrates a vector database with a large language model (LLM) to ensure the accuracy, reliability, and traceability of the generated technical answers.

Complete Inference Flow

When a user submits a technical query, the system executes a sequential pipeline comprising the following stages:

- **Reception and Embedding:** The incoming query via the REST API is encoded into a 1024-dimensional dense vector using the `BAAI/bge-m3` embedding model. The system dynamically allocates computational resources (CPU/CUDA) to optimize latency and throughput.
- **Context Retrieval:** The query vector is mapped against the Qdrant Cloud vector database. A *Cosine Similarity* search algorithm is applied to extract the top- K most semantically relevant documents (e.g., IBM Technotes and APAR records).
- **Contextualization:** The retrieved documents—specifically their `doc_id`, `title`, and `page_content`—are aggregated and formatted into a unified context block.
- **Text Generation:** Both the user’s query and the contextual block are injected into a customized *Chat Template*. The fine-tuned `Llama-3.2-3B-TechQA` model then acts as a synthesizer, analyzing the provided troubleshooting steps to generate a coherent, natural-language technical resolution.

Handling Unsuitable Context and Hardware Failures

To guarantee system robustness and ensure continuous availability in production environments, the pipeline implements comprehensive exception handling mechanisms:

- **Zero-Context Handling (Direct LLM Mode):** If the retrieved documents fall below a designated relevance threshold, or if the user explicitly disables the RAG module, the system bypasses context injection. Instead, it relies solely on the *parametric knowledge* acquired during the fine-tuning phase of the Llama model to provide a preliminary technical estimation.
- **Hardware Fallback (Graceful Degradation):** In real-world deployments, the generative language model may encounter operational failures, such as execution timeouts on standard CPUs or Out-Of-Memory (OOM) errors on constrained GPUs. To prevent complete system failure, a *Heuristic Parser* is immediately triggered. This rule-based fallback mechanism dynamically parses the raw structure of the retrieved documents (intelligently recognizing custom fields like `APAR ID:`, `Title:`, `Section:`, and markdown lists) and renders them directly into a user-friendly Markdown format. This guarantees that the user still receives vital troubleshooting information without experiencing a backend crash.

Source Referencing Mechanism

To fundamentally mitigate the "hallucination" problem inherent in generative models, every response generated by the system is accompanied by verifiable, traceable evidence:

- **Structured Response Delivery:** The backend API response payload (`AnswerResponse`) encapsulates not only the generated text but also an array of `SourceDocument` objects.
- **Confidence Quantification:** Each referenced document includes an explicitly exposed Similarity Score, allowing users to objectively evaluate how closely the retrieved literature aligns with their initial query.
- **Comprehensive Metadata Traceability:** Complete metadata—including the document ID, title, categorical section, and the original IBM URL—is synchronously returned. This architectural design enables the frontend interface to implement interactive features such as tooltips and direct hyperlinks, empowering IT engineers to seamlessly cross-reference the source documentation on the official provider's website.

5 Experimental Setup

5.1 Implementation Environment

The project uses two complementary execution environments. Model fine-tuning, embedding generation, corpus indexing, and benchmark inference were performed in Google Colab using a Linux-based Python 3 runtime and an NVIDIA Tesla T4 GPU with 14.56 GB of VRAM. CUDA acceleration was selected automatically when available; the notebooks otherwise expose a CPU fallback, although BGE-M3 embedding and Llama inference are substantially slower without a GPU. The T4 environment was chosen because it supports the 4-bit QLoRA workflow while remaining available in the resource-constrained Colab setting used by the project.

The deployable application is platform-independent at the source-code level. Its documented local requirements are Python 3.10 or newer for the engine and FastAPI backend, Node.js 18 or newer for the Vite frontend, and Docker when Qdrant is run as a separate service. Docker Compose exposes Qdrant’s HTTP/dashboard interface on port 6333 and its gRPC interface on port 6334, with a persistent named volume for vector storage. The React development server uses port 5173 and communicates with the Uvicorn/FastAPI server on port 8000.

Table 2 lists the principal software components recorded in the repository configuration files. Exact versions are reported where the project pins them; otherwise the table gives the minimum compatible version declared by the dependency specification.

Table 2: Implementation environment and principal software components

Component	Version or configuration	Role in the project
Compute environment	Google Colab, Linux, Python 3	Notebook execution, fine-tuning, indexing, and evaluation
GPU	NVIDIA Tesla T4, 14.56 GB VRAM	CUDA-accelerated QLoRA training, embedding, and model inference
Base generator	Llama 3.2-3B-Instruct, 4-bit	Transformer backbone for technical answer generation
Fine-tuning framework	Unsloth with QLoRA	Low-memory LoRA adapter training and model export
Embedding model	BAAI/bge-m3, 1024 dimensions	Normalized dense representations for questions and Technote chunks
Transformers stack	Transformers ≥ 4.40 , Accelerate ≥ 0.30 , BitsAndBytes ≥ 0.43	Model loading, quantization, and generation
Vector database	Qdrant Client ≥ 1.9 ; Qdrant Docker service	Cosine vector indexing, Top- k retrieval, metadata storage, and dashboard
RAG utilities	LangChain ≥ 0.2 , LangChain Community ≥ 0.2 , LangChain Qdrant ≥ 0.1	Text splitting and vector-store integration
Backend	FastAPI ≥ 0.111 , Uvicorn ≥ 0.29 , Pydantic ≥ 2.7	REST API, request validation, lifecycle management, and OpenAPI docs
Frontend	React 19.2.8, Vite 8.2.0, Axios 1.19.0	Chat interface, asynchronous API access, source display, and RAG controls
Data processing	Datasets 5.0.1, Pandas 3.0.5, PyArrow 25.0.0	TechQA loading, preprocessing, tabular analysis, and serialization
Evaluation	Evaluate ≥ 0.4 , ROUGE Score $\geq 0.1.2$, NLTK $\geq 3.8.1$	EM, token-level F1, ROUGE-L, and BLEU computation
Testing	Pytest ≥ 8.0 , Pytest-asyncio ≥ 0.23	Unit and API tests for the engine, retriever, generator, and backend routes

Configuration values are centralized in `engine/config.py` and can be overridden through environment variables. These include the embedding and LLM model identifiers, compute device, Qdrant host or cloud URL, API key, collection name, generation settings, and retrieval depth. This arrangement allows the same codebase to operate against a

local Docker deployment, Qdrant Cloud, or an in-memory vector-store fallback without changing the pipeline implementation.

5.2 Training Configuration

Fine-tuning was conducted for three epochs, corresponding to 171 optimization steps. A physical batch size of two examples and four gradient-accumulation steps produced an effective batch size of eight. Unsloth gradient checkpointing reduced activation-memory requirements, while the 8-bit AdamW optimizer reduced optimizer-state memory. On a Tesla T4 GPU with 14.563 GB of memory, training took 13.97 minutes and reached a peak reserved-memory footprint of 4.229 GB (29.04%). The complete configuration is reported in Table 3.

Table 3: Fine-tuning hyperparameters

Hyperparameter	Value
Base model	unsloth/Llama-3.2-3B-Instruct (4-bit NF4)
LoRA rank / alpha / dropout	16 / 16 / 0.0
Adapter target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Training / validation examples	450 / 160 (answerable only)
Learning rate	2×10^{-4}
Scheduler / warmup	Cosine decay / 10 warmup steps
Batch size / gradient accumulation	2 / 4 (effective batch size: 8)
Number of epochs / optimization steps	3 / 171
Maximum sequence length	2,048 tokens
Optimizer / weight decay	adamw_8bit / 0.01
Memory optimization	Unsloth gradient checkpointing
Loss masking	Assistant-response tokens only
Random seed	3407

5.3 Evaluation Metrics

To comprehensively evaluate our QA system, we employ distinct metrics for the retrieval module and the answer generation module.

Retrieval Metrics:

For the retrieval component, we evaluate the performance over the top- k retrieved results (where k represents the number of highest-ranked document chunks, e.g., $k \in \{1, 3, 5\}$).

- **Recall@ k (Hit Rate):** Measures the proportion of queries for which at least one relevant ground-truth document chunk is successfully retrieved within the top- k results.
- **Mean Reciprocal Rank (MRR):** Evaluates the system’s ranking quality by taking the average of the reciprocal ranks of the *first* relevant retrieved document chunk across all queries.

Answer Generation Metrics:

To assess the quality of the generated answers against human-annotated references, we utilize the following metrics:

- **Exact Match (EM):** A strict binary metric that yields 1 if the generated answer perfectly matches the reference answer after normalization, and 0 otherwise.
- **Token-level F1:** Computes the harmonic mean of token-level precision and recall, effectively rewarding partial matches and overlapping words between the generated and reference text.
- **BLEU:** Measures n-gram overlap between the generated text and the references, focusing primarily on precision while applying a penalty for overly short generated answers.
- **ROUGE-L:** Evaluates the Longest Common Subsequence (LCS) between the generated and reference answers, thereby capturing sentence-level structure and word order similarity.

Normalization Process:

Before calculating the aforementioned generation metrics, both the reference and generated answers undergo a strict normalization step to ensure a fair and robust comparison. This process includes converting all text to lowercase, removing punctuation marks, stripping standard English articles (e.g., "a", "an", "the"), and standardizing whitespaces to remove extra spaces or tabs.

6 Experimental Results and Analysis

6.1 Question Answering Results

The final end-to-end benchmark evaluates four system configurations on 160 answerable TechQA questions: the base and fine-tuned generators operating either without retrieval (standalone) or with evidence retrieved from the Qdrant knowledge base. The RAG configurations query an index containing 576,170 Technote chunks. Table 4 reports the official generation results together with the average response length of each configuration.

Table 4: End-to-end question-answering performance on the 160-question TechQA benchmark. Metric values are percentages.

Configuration	EM	F1	ROUGE-L	BLEU-4	Avg. words
Base Model (Standalone)	0.00%	15.00%	10.60%	1.10%	80.0
Fine-tuned Model (Standalone)	0.00%	22.30%	19.00%	5.90%	35.0
Base Model + RAG	0.00%	18.83%	13.75%	1.78%	45.0
Fine-tuned Model + RAG	0.00%	19.53%	16.09%	2.25%	33.9

Effect of retrieval augmentation. RAG improves the base model’s F1 from 15.00% to 18.83% (25.5% relative) and ROUGE-L from 10.60% to 13.75%. Its average response length also decreases from 80.0 to 45.0 words. These results show that evidence from the 576,170-chunk Qdrant index helps the unadapted model answer specialized technical questions more accurately and concisely.

Effect of QLoRA domain adaptation. QLoRA produces the strongest configuration in the standalone setting. F1 rises from 15.00% to 22.30% (48.7% relative), ROUGE-L from 10.60% to 19.00%, and BLEU-4 from 1.10% to 5.90% (approximately 5.3 times higher). This improvement indicates successful adaptation to IBM terminology and troubleshooting patterns, while the reduction to 35.0 words per answer better matches the concise reference style.

Fine-tuned Model + RAG reaches 19.53% F1 and averages 33.9 words, closely matching the 34-word ground-truth median. Its lower F1 than Fine-tuned Standalone (22.30%) indicates that retrieval quality remains the main bottleneck, as discussed in Section 6.3.1. Exact Match remains 0.00% because generative answers often differ from references in punctuation or equivalent wording; F1, ROUGE-L, and BLEU-4 therefore provide more informative measures of partial agreement.

6.2 Retrieval Results

Table 5: Retrieval performance on the validation set ($N = 20$)

Configuration	Recall@1	Recall@3	Recall@5	MRR@5
bge-m3 dense retrieval	15.00%	25.00%	40.00%	0.2300

Analysis:

The baseline *bge-m3 dense retrieval* configuration achieves a Recall@5 of 40.00% and

an MRR@5 of 0.2300. While the model demonstrates the ability to retrieve relevant context for a portion of the queries, the relatively low MRR indicates that the exact ground-truth documents are often ranked lower in the top-5 results rather than appearing at the very top. A qualitative error analysis of the missed queries reveals that the dataset contains highly technical IT queries with specific error codes (e.g., "Error #2070") and exact library names (e.g., "jextract"). Purely semantic dense embeddings like *bge-m3* often struggle with these exact-match constraints. Due to time limitations in this phase, this baseline dense retrieval is adopted as our current selected configuration. In future iterations, implementing a Hybrid Search approach (combining dense embeddings with BM25 lexical search) and utilizing a Cross-Encoder for re-ranking would be expected to significantly improve both Recall@1 and MRR.

6.3 Qualitative Analysis and Error Analysis

6.3.1 Generator Error Analysis

The main failure mode is *distractor context degradation*. Adding retrieval reduces the fine-tuned model's F1 from 22.30% to 19.53%. Qdrant achieves an Answer-in-Context@1 of only 29.4% (47/160), so the top-ranked chunk lacks sufficient answer evidence for 70.6% of the questions. Although Answer-in-Context@10 reaches 68.1% (109/160), relevant evidence is frequently ranked below misleading or incomplete passages. The extraction-oriented prompt then encourages the generator to follow this distractor instead of relying on the domain knowledge acquired through fine-tuning.

Limiting generation to `max_new_tokens=64` effectively controls verbosity: Fine-tuned Model + RAG produces an average of 33.9 words, corresponding to 99.7% of the 34-word ground-truth median. The remaining performance loss is therefore primarily attributable to retrieval quality rather than answer length.

Future work should combine BGE-M3 dense retrieval with BM25 lexical search and a cross-encoder reranker. Improving Answer-in-Context@1 would reduce distractor exposure and allow the fine-tuned generator to use external Technote evidence more reliably.

6.3.2 Retrieval Error Analysis

Our baseline retrieval system missed the correct document in 12 out of 20 queries. By looking at these failed cases, we identified several reasons why the system returns irrelevant, incomplete, or misleading chunks.

1. Embedding Similarity vs. Exact Keywords: Dense embedding models (like *bge-m3*) are good at understanding general meanings, but they struggle with exact keyword matching. In our dataset, users often search for exact technical terms or error codes (e.g., "Error #2070" or "*SqlIntegrityConstraintViolation*"). The model retrieves documents that

talk about similar errors because their **embedding similarity** is high, but it misses the exact specific error the user actually needs.

2. Chunking Issues: The way we cut documents into smaller pieces (**chunking**) also causes errors. If a chunk is too large, important keywords like error codes get lost in the long text. If a chunk is too small, it might contain the error code but cut off the actual solution. This causes the retriever to return incomplete or misleading chunks that don't help answer the question.

3. Query Ambiguity: Some user questions are too short or lack context (**query ambiguity**). For example, a query like *"Which Socket Gateway should I be using with Netcoo..."* does not provide enough details about the system version. Because the question is unclear, the model just guesses and returns generic gateway documents instead of the specific right answer.

4. Missing Metadata Filters: Currently, our system only searches by comparing text vectors. It does not use **metadata** like product name, software version, or category. When a user asks a question about a specific database like *"BPMDB"*, the system might return irrelevant documents about other databases just because they share similar technical words.

5. Selected Retrieval Threshold ($k = 5$): We evaluated the system by looking only at the top 5 results (the **selected retrieval threshold**). Because many technical support documents are very similar to each other, the model often finds the correct document but ranks it at position 6 or 10. By keeping the threshold as low as 5 without using a re-ranker, the relevant document is missing from the top- k results.

7 Web-based Application

7.1 Application Architecture

The application follows a layered architecture that separates presentation, API handling, and machine-learning logic. The frontend is implemented with React and Vite and runs on port 5173. It communicates through HTTP/JSON with a FastAPI backend served by Uvicorn on port 8000. The backend validates requests using Pydantic schemas and delegates question-answering operations to a service layer, which in turn invokes the QA engine. The engine encodes queries with BGE-M3, retrieves the top- k chunks from Qdrant, constructs the context-augmented prompt, and requests an answer from the fine-tuned Llama 3.2-3B generator. Qdrant runs as a separate Docker service on ports 6333 and 6334 and stores the dense vectors and document metadata.

This separation keeps the user interface independent of model implementation details and allows the retrieval and generation components to be tested through the API. The principal endpoints, all registered under the `/api` prefix, are as follows:

- `POST /api/ask` accepts a question, `top_k`, temperature, retrieval mode, and an optional session identifier. It returns the generated answer, source documents, similarity scores, model information, confidence score, and end-to-end latency.
- `GET /api/health` reports the API, Qdrant, embedding-model, and generator status, together with the active device, engine mode, and number of indexed documents.
- `POST /api/index` triggers document indexing or re-indexing for the requested corpus split and returns the collection name, number of indexed items, and indexing duration.
- `GET /api/collections` returns Qdrant collection statistics; `GET /api/sample-questions` supplies curated demonstration queries; and `GET /api/metrics` exposes the model-comparison results used by the evaluation dashboard.

When the complete GPU/Qdrant engine is unavailable, the service can fall back to a small in-memory demonstration corpus. This graceful-degradation mode preserves the API contract and allows the frontend workflow to be demonstrated, although it is not equivalent to evaluation with the complete indexed corpus and fine-tuned generator.

7.2 User Interface and Main Features

The interface is organized into three main regions. The central *ChatWindow* presents the conversation, renders generated Markdown, shows the latency of each response, and provides a copy button. Questions can be typed into the expandable input field and submitted with the send button or the Enter key. Before a conversation begins, the same area displays several suggested TechQA questions for rapid testing.



Figure 2: ChatWindow for submitting questions and reading generated answers.

The left *Sidebar* exposes the principal RAG controls. Users may choose dense retrieval, hybrid retrieval, or direct LLM generation; adjust the number of retrieved documents from one to ten; and set the sampling temperature. It also lists categorized sample questions covering WebSphere, DB2, IBM MQ, security, system tuning, and Transformer training. These controls make the web application useful not only as a demonstration interface but also as an interactive tool for comparing inference configurations.

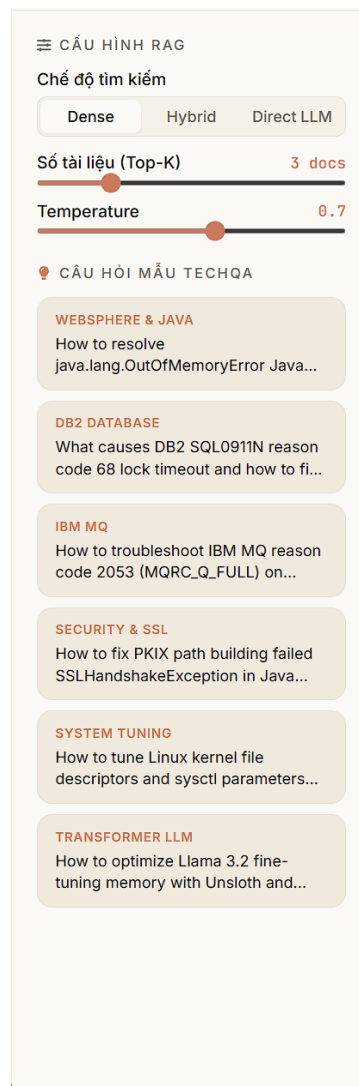


Figure 3: Sidebar for configuring RAG and selecting sample questions.

Each generated response includes clickable source buttons labelled with the Technote identifier, title, and similarity score. Selecting one opens the right-hand *Document Inspector*. This panel displays the document category, title, identifier, full retrieved content, normalized similarity score, and a link to the original article when a URL is available. Consequently, the user can compare the generated answer directly with its supporting evidence instead of treating the model output as an unverifiable statement.

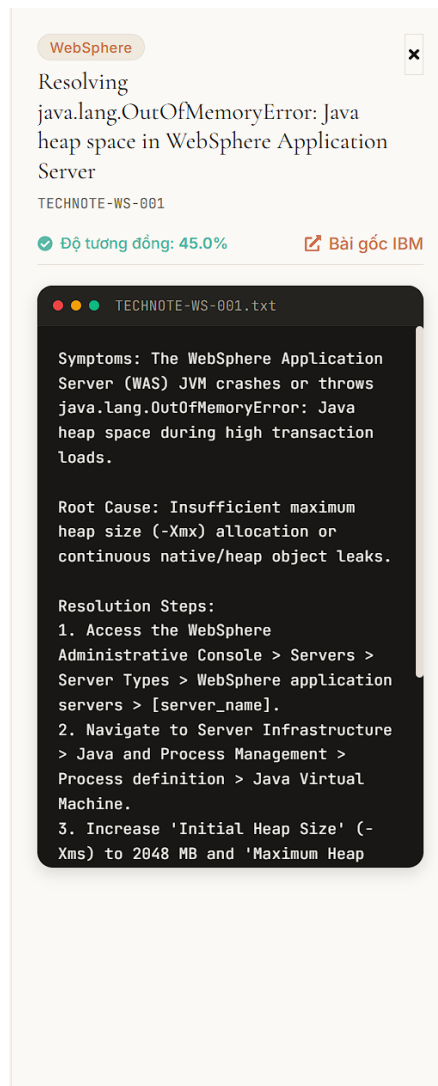


Figure 4: Document Inspector for verifying retrieved evidence.

7.3 Deployment and Demonstration

The demonstration is deployed on a local machine as three cooperating processes. After configuring the environment variables, Qdrant is started from the project root using Docker:

```
docker compose up -d
```

The FastAPI backend and React development server are then started in separate terminals:

```
uvicorn backend.main:app --reload --port 8000
```

```
cd frontend
```

```
npm install
```

```
npm run dev
```

The user interface is available at <http://localhost:5173>, the API at <http://localhost:8000>, the interactive OpenAPI documentation at <http://localhost:8000/docs>, and the Qdrant dashboard at <http://localhost:6333/dashboard>. The frontend checks system health when it is loaded and repeats the check every 30 seconds. Its API client permits a long request timeout because loading and running the language model can be much slower than ordinary REST operations.

Response time is measured inside the backend for every request and returned as `latency_ms`; the ChatWindow displays this value with the corresponding answer. The benchmark metadata currently exposed by `/api/metrics` reports an average latency of 210 ms per query for the Fine-tuned Model + RAG configuration. This software-reported value should be interpreted together with the execution mode: actual local latency depends strongly on whether the full model is already loaded, whether CUDA is available, the selected Top- k , and whether the live Qdrant pipeline or the lightweight demonstration fallback is active.

The main deployment limitation is hardware capacity. Live inference is intended for a CUDA-capable GPU, while the 3.21-billion-parameter generator is loaded in 4-bit form to reduce VRAM consumption. Similarly, the approximately 800,000 original Technotes cannot all be processed comfortably under the project's limited compute and vector-storage budget; corpus indexing is therefore bounded and performed in batches. On a CPU-only local machine, model loading and text generation may be substantially slower or may trigger the fallback service. Despite these constraints, the local deployment demonstrates the complete user workflow: system monitoring, configurable retrieval, grounded answer generation, source inspection, and access to evaluation metrics.

Team Contributions

Team member responsibilities

Student ID	Name	Tasks	Share
23127366	Võ Lê Ngọc Hiếu	Section 1; Sections 4.1 and 5.1; and all of Section 7. Responsible for designing, implementing, and documenting the web-based application and its system architecture.	100%
23127386	Nguyễn Duy Khánh	Sections 2.1 and 2.3; Section 3.3.2; Sections 4.2, 4.3, and 4.5; Section 5.3; Sections 6.2 and 6.3.2. Responsible for Technote corpus preprocessing, vector indexing, Qdrant integration, retrieval experiments, and retrieval error analysis.	100%
23127425	Tăng Nhật Minh	Section 2.2; Sections 3.1, 3.2, 3.3.1, and 3.4; Section 4.4; Section 5.2; Sections 6.1 and 6.3.1. Responsible for QLoRA fine-tuning, dataset analysis, generator configuration, the four-configuration comparative benchmark, and generator error analysis.	100%

References

- [1] E. Castelli et al., “TechQA: A Real-World Benchmark for AI Question Answering Using IBM Technotes,” 2020.
- [2] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” 2020.
- [3] J. Chen et al., “BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings,” 2024.
- [4] Y. Ding et al., “A Survey on RAG Meets LLMs: Towards Retrieval-Augmented Large Language Models,” 2024.
- [5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” in *Advances in Neural Information Processing Systems*, 2023.
- [6] A. Dubey et al., “The Llama 3 Herd of Models,” arXiv preprint arXiv:2407.21783, 2024.